Java Condition Order – How the Order of Conditions Changes Program Behavior

---

Learning Objectives

After this topic, you will be able to:

- Understand why the order of conditions matters.
- Learn how Java evaluates conditions.
- Avoid logical errors caused by incorrect condition order.
- Write safer and more efficient if statements.

---

1. Why Does Condition Order Matter?

Java evaluates conditions **from left to right**.

With && and ||, Java uses **short-circuit evaluation**, meaning it may stop checking further conditions once the result is already known.

Changing the order of conditions can affect:

- Program output
- Performance
- Runtime errors (like NullPointerException)

---

2. Example: Safe Null Check

✅ Correct Order

```
String name = null;

if (name != null && name.length() > 3) {
    System.out.println("Valid Name");
}
```

No exception occurs because Java first checks name != null.

❌ Wrong Order

```
String name = null;

if (name.length() > 3 && name != null) {
    System.out.println("Valid Name");
}
```

Result

```
NullPointerException
```

Reason: name.length() is executed before checking if name is null.

---

3. Example: Using ||

Correct

```
boolean isAdmin = true;
boolean hasPermission = false;

if (isAdmin || hasPermission) {
    System.out.println("Access Granted");
}
```

Since isAdmin is true, Java doesn't evaluate the second condition.

---

4. Example: Range Checking

❌ Wrong Order

```
int marks = 95;

if (marks >= 50) {
    System.out.println("Pass");
} else if (marks >= 90) {
    System.out.println("
```